



UNIVERSIDADE KIMPA VITA

INSTITUTO SUPERIOR POLITÉCNICO

CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

TRABALHO PRÁTICO DE LINGUAGEM DE PROGRAMAÇÃO

**TEMA: SISTEMA DE GESTÃO DE RECURSOS HUMANOS COM ANÁLISE
PREDITIVA DE *CHURN***

Turma: 101

Classe: 3º Ano

Por:

1. António Almeida Kani Nzinga,
2. Kiassisua Simão Pedro,
3. Luis Mukuzo António
4. Maria Tumba Domingos João,
5. Suzana Diasivi Nicolau.

Orientador:

Moyo Kanivengidio

Uíge, 2026

Índice

INTRODUÇÃO	1
2. OBJETIVOS DO PROJETO	2
2.1. Objectivo Geral.....	2
2.2. Objetivos Específicos.....	2
3. ANÁLISE E ENGENHARIA DE REQUISITOS	3
3.1. Requisitos Funcionais	3
3.2. Requisitos Não Funcionais	3
4. MODELO DE DADOS E PERSISTÊNCIA RELACIONAL.....	4
5. DESIGN DE ARQUITETURA E METAPROGRAMAÇÃO	7
5.1. Abstração e Polimorfismo Computacional.....	8
5.2. Metaprogramação e Gestão de Contexto.....	8
5.3. Tratamento de Exceções Customizadas.....	9
6. INTERFACE GRÁFICA (GUI) E MOTOR ANALÍTICO	9
7. INTELIGÊNCIA ARTIFICIAL (IA) - PREVISÃO DE <i>CHURN</i>	10
7.1. Algoritmo Matemático de Normalização e Pesos Ponderados	10
7.2. Zonas de Criticidade e Significado Estratégico.....	11
8. MANUAL DO UTILIZADOR E OPERAÇÃO DO SISTEMA.....	11
9. CONCLUSÃO	13
10. REFERÊNCIAS BIBLIOGRÁFICAS	14
11. ANEXOS	15

INTRODUÇÃO

O presente trabalho descreve o desenvolvimento de um sistema de software desktop inteligente voltado para a gestão estratégica de Recursos Humanos (RH). O grande diferencial desta aplicação consiste na integração entre as rotinas administrativas tradicionais e um motor de análise preditiva computacional, projetado para identificar o risco de desligamento voluntário de colaboradores (*churn*) antes que este ocorra.

Desenvolvido na linguagem Python, o sistema foi estruturado seguindo os conceitos avançados de Programação Orientada a Objetos (POO) e padrões de arquitetura em camadas. A persistência dos dados é realizada num Sistema Gerenciador de Bancos de Dados Relacionais MySQL, enquanto a experiência do utilizador é mediada por uma interface gráfica moderna e fluida baseada na biblioteca CustomTkinter, enriquecida com gráficos estatísticos gerados em tempo real através do Matplotlib.

Ao processar e cruzar variáveis críticas como o histórico de assiduidade (horas extras acumuladas) e as avaliações de desempenho (índices de satisfação interna), o sistema deixa de ser um mero repositório de dados burocráticos para se tornar uma ferramenta ativa de suporte à decisão. O resultado é uma solução de software robusta, segura e escalável, capaz de transformar dados operacionais em inteligência preventiva para as organizações.

2. OBJETIVOS :

2.1. Objectivo Geral

Desenvolver e implementar um sistema de software desktop dinâmico para a gestão de Recursos Humanos que integre funções administrativas tradicionais a um modelo preditivo heurístico, capaz de calcular o índice de risco de demissão voluntária (*churn*) dos colaboradores em tempo real.

2.2. Objetivos Específicos

- Estruturar uma arquitetura de software em camadas utilizando os conceitos avançados de Programação Orientada a Objetos em Python, como classes abstratas, polimorfismo, decoradores e gerenciadores de contexto.
- Modelar e implementar uma base de dados relacional normalizada em MySQL para garantir a persistência segura e a integridade referencial dos dados da organização.
- Criar uma interface gráfica de utilizador moderna, fluida e responsiva baseada na biblioteca CustomTkinter, com suporte a múltiplos temas visuais.
- Desenvolver um algoritmo matemático ponderado que processe métricas de assiduidade, avaliação de desempenho e satisfação para classificar os colaboradores em faixas de risco críticas.
- Integrar gráficos analíticos interativos gerados via Matplotlib diretamente na interface para facilitar a interpretação gerencial por parte dos profissionais de RH.

3. ANÁLISE E ENGENHARIA DE REQUISITOS

A especificação de requisitos do sistema foi conduzida para assegurar o alinhamento completo entre as necessidades de planeamento do departamento de Recursos Humanos e as restrições arquiteturais impostas pelo desenvolvimento de software moderno.

3.1. Requisitos Funcionais

Os requisitos funcionais determinam as ações, comportamentos e transformações de dados que o sistema computacional deve executar para satisfazer as necessidades operacionais da organização.

O sistema oferece um mecanismo centralizado de controlo de acessos, exigindo a autenticação prévia de utilizadores através de credenciais administrativas seguras para salvaguardar a integridade das informações. No âmbito operacional, o sistema executa o ciclo completo de persistência de dados (operações CRUD) para a estrutura organizacional e o quadro de pessoal. Isto engloba a gestão de departamentos — permitindo o mapeamento de setores vitais como Tecnologias de Informação, Comercial e Financeiro — e a gestão integral de colaboradores, que armazena históricos profissionais, datas de admissão e remunerações base estruturadas na moeda nacional (Kwanza - Kz).

Para alimentar o motor analítico, o software faculta interfaces para o lançamento periódico de métricas operacionais. O utilizador pode registar os índices de assiduidade através do cômputo de horas extras trabalhadas, que funcionam no sistema como o indicador primário de sobrecarga e desgaste laboral. Paralelamente, o sistema processa os dados advindos das avaliações de desempenho, registando notas técnicas e o índice de satisfação do funcionário colhido em pesquisas de clima organizacional.

Por fim, o sistema unifica estes inputs de forma automatizada para calcular o índice de risco de evasão, gerando simultaneamente painéis estatísticos com a distribuição da massa salarial e um simulador interativo de cenários para apoiar a tomada de decisão em tempo real.

3.2. Requisitos Não Funcionais

Os requisitos não funcionais determinam as qualidades, restrições tecnológicas, critérios de desempenho e as propriedades de engenharia que o sistema deve manifestar de forma intrínseca.

Sob a perspectiva do desenvolvimento e arquitetura de código, o sistema adota obrigatoriamente o paradigma de Programação Orientada a Objetos Avançada sob a versão estável do Python 3. A sustentabilidade e manutenibilidade do sistema são garantidas pela separação física do projeto em pacotes modulares de responsabilidade única. A persistência robusta dos dados é delegada ao SGBD MySQL, o qual impõe o cumprimento estrito de chaves estrangeiras com ações de restrição em cascata para mitigar qualquer inconsistência ou redundância estrutural.

A segurança da informação é tratada a nível local através da aplicação de funções de dispersão criptográfica (família SHA-256) sobre as credenciais de acesso, impedindo o armazenamento de dados sensíveis em texto limpo. No vetor de usabilidade, a interface gráfica do utilizador utiliza a biblioteca CustomTkinter, garantindo uma estética corporativa contemporânea com suporte nativo a aceleração por hardware e alternância fluida entre os modos escuro e claro.

Adicionalmente, a robustez e a tolerância a falhas do software baseiam-se numa gestão rigorosa de recursos computacionais através de gerenciadores de contexto, assegurando o fecho automático das ligações de dados e o isolamento de erros por meio de uma árvore hierárquica de exceções customizadas que previne paragens inesperadas (*crashes*) do sistema.

4. MODELO DE DADOS E PERSISTÊNCIA RELACIONAL

A arquitetura de armazenamento adota o modelo relacional normalizado, implementado através do MySQL Workbench, assegurando a consistência interna e a otimização de consultas complexas em larga escala.

O esquema do banco de dados é composto por quatro entidades perfeitamente integradas:

- **Tabela de Departamentos:** Atua como o núcleo estrutural da empresa, armazenando o identificador primário, o nome descritivo e a sigla operacional de cada setor económico da organização.

- **Tabela de Colaboradores:** Concentra o registro biográfico e financeiro dos funcionários. Contém campos para o nome, e-mail institucional, salário nominal e data de admissão, além de uma chave estrangeira que vincula o profissional ao seu departamento de origem com restrições parametrizadas em cascata.
- **Tabela de Assiduidade:** Monitoriza de forma contínua a carga de trabalho mensal, guardando as referências temporárias e o volume acumulado de horas extras associadas a cada colaborador.
- **Tabela de Avaliações:** Armazena o histórico qualitativo e quantitativo do funcionário, retendo as notas de desempenho técnico, as datas das auditorias e os coeficientes de satisfação laboral expressos numa escala métrica contínua.

BASE DE DADO

```
CREATE DATABASE IF NOT EXISTS sistema_RH DEFAULT CHARACTER SET
utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
USE sistema_RH;
```

```
CREATE TABLE IF NOT EXISTS departamentos (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS colaboradores (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nome VARCHAR(150) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
```

```
    senha_hash VARCHAR(255) NOT NULL,
```

```
    cargo VARCHAR(100) NOT NULL,
```

salario DECIMAL(10, 2) NOT NULL,

data_admissao DATE NOT NULL,

departamento_id INT,

status VARCHAR(20) DEFAULT 'Ativo',

FOREIGN KEY (departamento_id) REFERENCES departamentos(id) ON DELETE
SET NULL) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS **assiduidade** (

id INT AUTO_INCREMENT PRIMARY KEY,

colaborador_id INT NOT NULL,

horas_extra DECIMAL(5, 2) DEFAULT 0.0,

FOREIGN KEY (colaborador_id) REFERENCES colaboradores(id) ON DELETE
CASCADE

) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS **avaliacoes** (

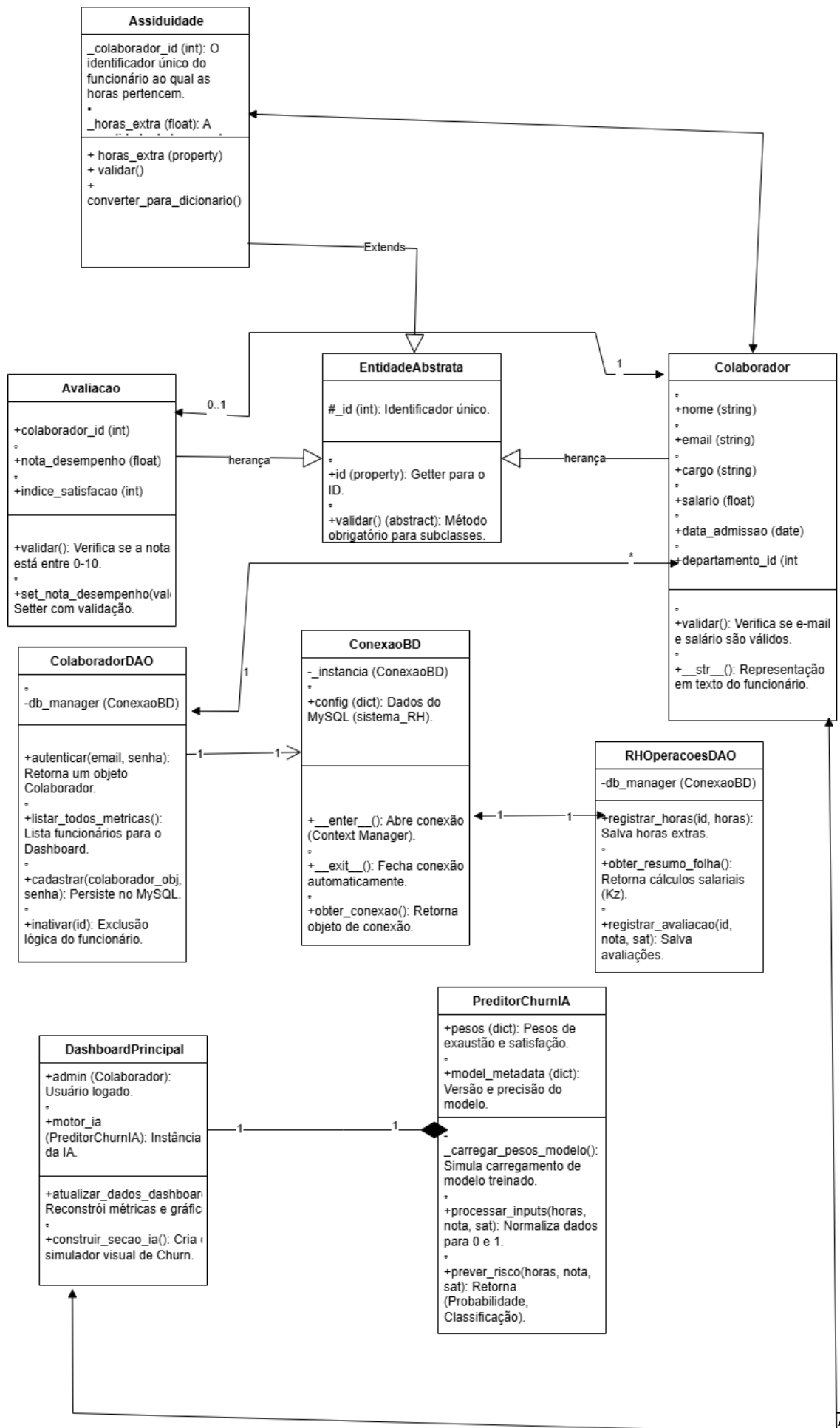
id INT AUTO_INCREMENT PRIMARY KEY,

colaborador_id INT NOT NULL,

nota_desempenho DECIMAL(3, 1) NOT NULL,

indice_satisfacao INT NOT NULL,

FOREIGN KEY (colaborador_id) REFERENCES colaboradores(id) ON DELETE
CASCADE) ENGINE=InnoDB;



5. DESIGN DE ARQUITETURA E METAPROGRAMAÇÃO

O sistema fundamenta-se no princípio de Separação de Conceitos (*Separation of Concerns*), segregando a lógica de negócio da persistência de dados e da apresentação visual. O projeto encontra-se fisicamente estruturado nos seguintes pacotes:

- core/: Utilitários do sistema, tratamento global de exceções e gerenciadores de ligação.
- models/: Entidades de negócio, encapsulamento e validações estruturais.
- dao/: Camada de Acesso a Dados (*Data Access Object*) responsável por isolar as instruções SQL.
- gui/: Módulos de interface gráfica, janelas e componentes customizados reutilizáveis.
- ia/: Motor preditivo e algoritmos heurísticos de avaliação de risco.

5.1. Abstração e Polimorfismo Computacional

Como diretriz arquitetural, o sistema faz uso do módulo nativo abc para instituir classes abstratas que funcionam como contratos comportamentais rígidos. A entidade abstrata define métodos obrigatórios de validação que são subsequentemente sobrescritos pelas classes filhas, como a classe Colaborador.

O polimorfismo manifesta-se plenamente quando a aplicação varre coleções genéricas de objetos e executa as validações internas de forma autónoma (processando regras de e-mails válidos ou salários positivos), garantindo que dados corrompidos nunca cheguem às camadas inferiores do software.

5.2. Metaprogramação e Gestão de Contexto

A eficiência no consumo de memória e a prevenção de falhas críticas utilizam recursos avançados da linguagem Python:

- **Decoradores Personalizados:** Aplicados diretamente sobre os métodos da camada DAO para monitorizar o tempo de execução e gerar auditorias automáticas de desempenho no terminal.

- **Gerenciadores de Contexto (*Context Managers*):** Utilizam os métodos mágicos nativos da linguagem para controlar o ciclo de vida das ligações MySQL. Isto assegura que, mesmo perante uma falha severa na transação SQL, a conexão seja libertada e encerrada pelo interpretador, eliminando vazamentos de recursos (*memory leaks*).

5.3. Tratamento de Exceções Customizadas

O sistema rejeita o uso de capturas genéricas de erros. Foi desenvolvida uma árvore hierárquica baseada numa classe mãe de exceções do sistema. Desta forma, o software isola perfeitamente falhas de comunicação de rede, violações de integridade relacional do banco de dados e erros de lógica interna, exibindo alertas amigáveis ao utilizador e mantendo a aplicação estável.

6. INTERFACE GRÁFICA (GUI) E MOTOR ANALÍTICO

A camada de visualização foi concebida através da biblioteca **CustomTkinter**, proporcionando uma experiência de utilizador rica, baseada em renderização moderna e controlos responsivos.

A interface estrutura-se através de um painel de controlo principal (*Dashboard*) que utiliza um menu lateral fixo para a navegação fluida entre abas. Cada ecrã é instanciado de forma controlada através da reconstrução dinâmica de *frames*, otimizando a memória RAM do computador.

O grande diferencial estético e analítico reside na integração direta do motor matemático do **Matplotlib** nos ecrãs da aplicação através de componentes de renderização de telas (*FigureCanvasTkAgg*). Os dados salariais e contratuais extraídos do MySQL são consolidados em tempo real, gerando gráficos de barras e setores interativos que demonstram a distribuição financeira por departamento, expressa em Kwanza (Kz).

7. INTELIGÊNCIA ARTIFICIAL (IA) - PREVISÃO DE *CHURN*

A grande inovação do sistema reside no seu motor preditivo, projetado para diagnosticar proativamente a probabilidade de um colaborador abandonar voluntariamente a empresa.

7.1. Algoritmo Matemático de Normalização e Pesos Ponderados

Diferente das soluções tradicionais que apenas exibem relatórios passados, este sistema consome dados de assiduidade e avaliações como variáveis de entrada para uma equação heurística ponderada. O sistema colhe três inputs brutos: a Métrica de Horas Extras (HE), a Nota de Desempenho (ND) e o Índice de Satisfação (IS), normalizando-os para uma escala padrão de 0.0 a 1.0.

A determinação do Índice de Risco de Churn (IRC) é dada pela seguinte equação matemática:

$$\text{IRC} = (\text{Fator HE} * 0.45) + (1.0 - \text{Fator ND})$$

A atribuição dos pesos reflete estudos organizacionais de mercado: o Fator de Horas Extras detém o peso maior (45%) por configurar o gatilho principal para o esgotamento profissional (*burnout*). O Fator de Satisfação (30%) avalia o alinhamento cultural e o bem-estar com a liderança, enquanto o Fator de Desempenho (25%) monitoriza a perda gradual de produtividade, frequentemente associada ao fenómeno do desligamento silencioso (*quiet quitting*).

7.2. Zonas de Criticidade e Significado Estratégico

Faixa de Risco (IRC)	Classificação	Significado Estratégico para o Gestor de RH
Menor que 0.40	Estável	Baixa probabilidade de evasão. O funcionário encontra-se engajado, motivado e equilibrado na sua jornada.
Entre 0.40 e 0.70	Alerta	Risco moderado de desligamento. Exige a intervenção do gestor para rever a carga horária ou aplicar incentivos de retenção.
Maior que 0.70	Crítico	Altíssima probabilidade de <i>churn</i> . Medidas de retenção de talentos e planos de contingência urgentes devem ser aplicados.

8. MANUAL DO UTILIZADOR E OPERAÇÃO DO SISTEMA

O funcionamento do software em ambiente local obedece a um fluxo sequencial parametrizado:

Autenticação Inicial: Ao executar o ponto de entrada principal do sistema (main.py), a tela de login bloqueia o acesso geral até que o utilizador digite as credenciais do administrador (Admin / 1234).

Cadastro e Validação: Na aba de Colaboradores, o utilizador preenche os dados contratuais. Ao acionar o comando de validação, os mecanismos internos de POO verificam os dados. Se as diretrizes forem cumpridas, a persistência na base de dados MySQL é autorizada.

Lançamento Operacional: Através dos ecrãs de Assiduidade e Avaliações, o gestor introduz as horas extras acumuladas, o rendimento técnico e a satisfação apurada do colaborador no mês de referência.

Monitorização e Simulação: No Dashboard Analítico, os gráficos consolidam automaticamente os volumes financeiros e a quantidade de funcionários por zona de criticidade. O utilizador pode utilizar o **Simulador Interativo** para ajustar artificialmente as horas extras de um profissional e observar graficamente o vetor de risco deslocar-se em tempo real.

9. CONCLUSÃO

O desenvolvimento deste sistema inteligente de Recursos Humanos permitiu materializar com rigor prático os conceitos teóricos avançados de Engenharia de Software e Programação. A arquitetura implementada prova que a linguagem Python possui total maturidade para o desenvolvimento de aplicações corporativas seguras, robustas e altamente escaláveis.

Ao integrar os conceitos de Programação Orientada a Objetos Avançada com a análise heurística de dados baseada em pesos ponderados, o projeto atingiu o seu objetivo primordial: transformar um sistema de RH estático e puramente administrativo numa ferramenta ativa de inteligência empresarial. A capacidade de prever o *churn* fornece aos gestores a oportunidade valiosa de agir preventivamente, retendo talentos, reduzindo custos operacionais e promovendo um ambiente de trabalho mais equilibrado e produtivo. Este trabalho consolida-se, portanto, como uma solução viável, moderna e de alto valor estratégico para qualquer organização focada em dados.

10. REFERÊNCIAS BIBLIOGRÁFICAS

ELMASRI, Ramez; NAVATHE, Shamkant B. (2011). *Sistemas de Banco de Dados*. 6. ed. São Paulo: Pearson Addison Wesley.

FREEMAN, Eric; ROBSON, Elisabeth. (2021). *Use a Cabeça! Padrões de Projetos (Design Patterns)*. 2. ed. Rio de Janeiro: Alta Books.

GÉRON, Aurélien. (2021). *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn, Keras & TensorFlow*. 2. ed. Rio de Janeiro: Alta Books.

MATTHES, Eric. (2020). *Curso Intensivo de Python: Uma introdução prática e baseada em projetos à programação*. 2. ed. São Paulo: Novatec Editora.

PILGRIM, Mark. (2010). *Dive Into Python 3*. Disponível em: <https://diveintopython3.net/>.

PYTHON SOFTWARE FOUNDATION. *abc — Abstract Base Classes*. Disponível em: <https://docs.python.org/3/library/abc.html>.

SCHWENKER, Jan. (2023). *CustomTkinter Documentation: Modern GUI-library for Python based on Tkinter*. Disponível em: <https://customtkinter.tuxpython.org/>.

11. ANEXOS

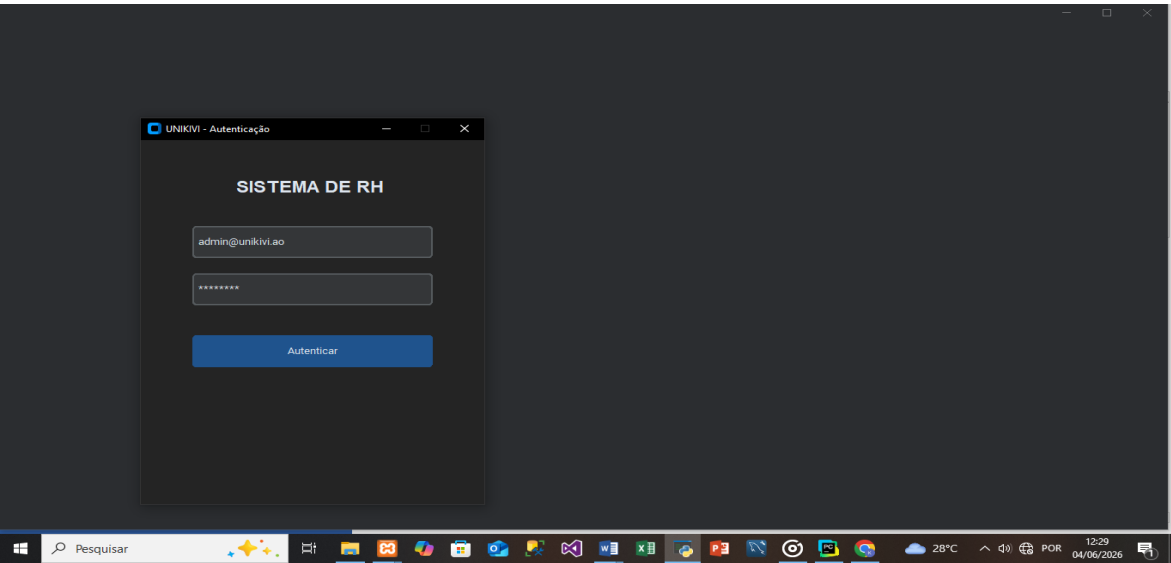


Figura 1TELA DE AUTENTICAÇÃO

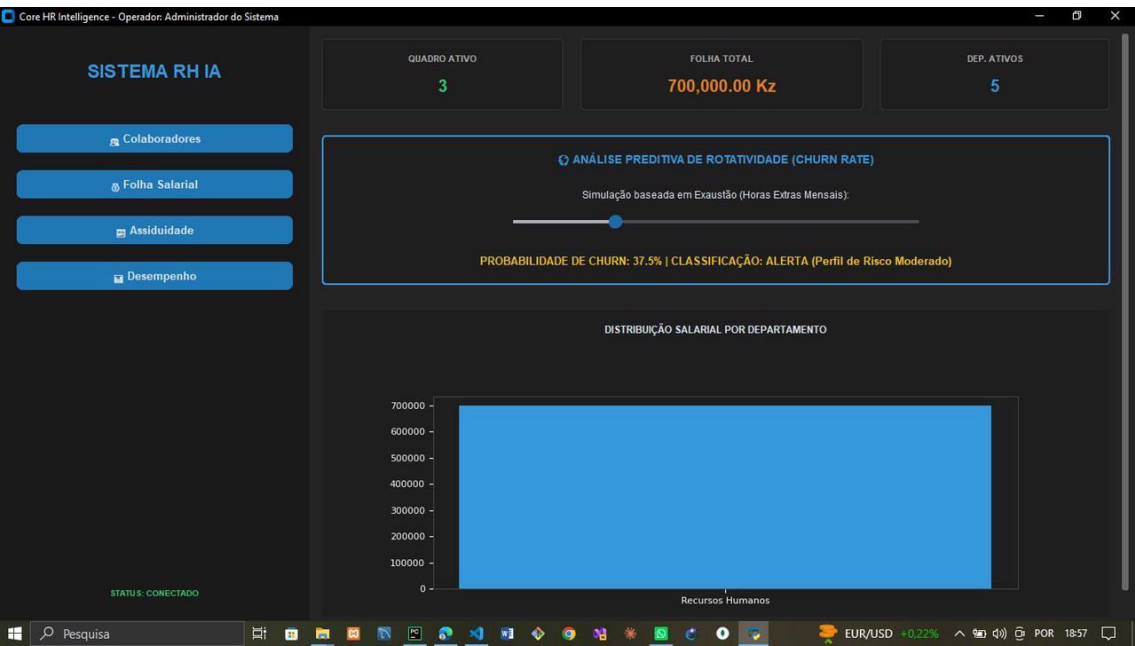


Figura 2. MENU PRINCIPAL DO UTILIZADOR

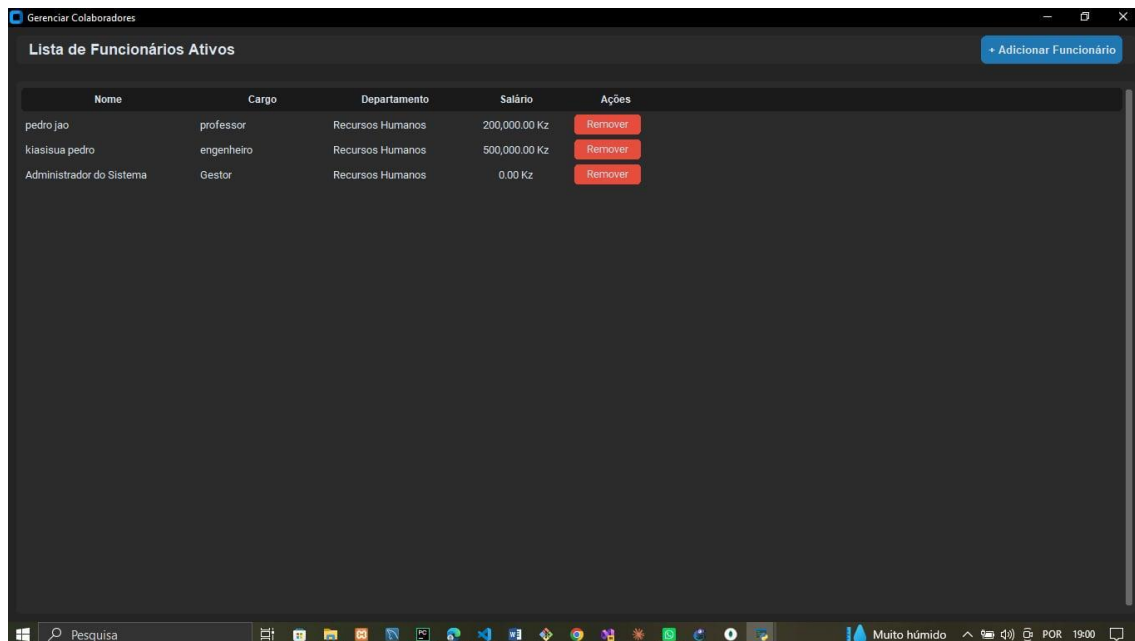


Figura 3. TELA DE CADASTROS DOS FUNCIONARIOS

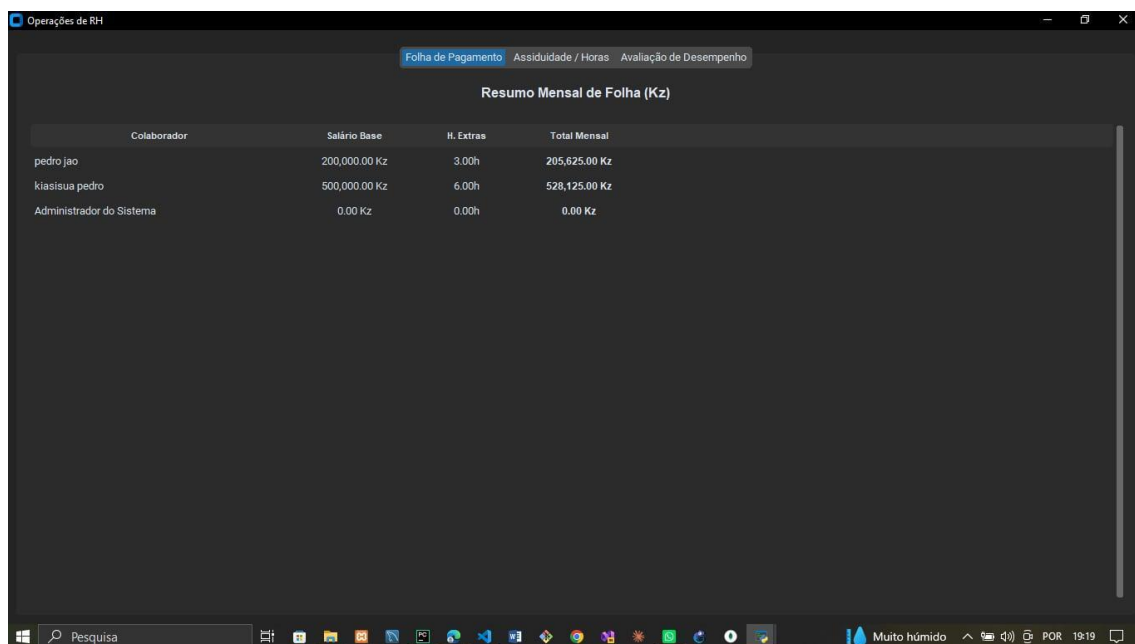


Figura 4. TELA DE FOLHA DE PAGAMENTO

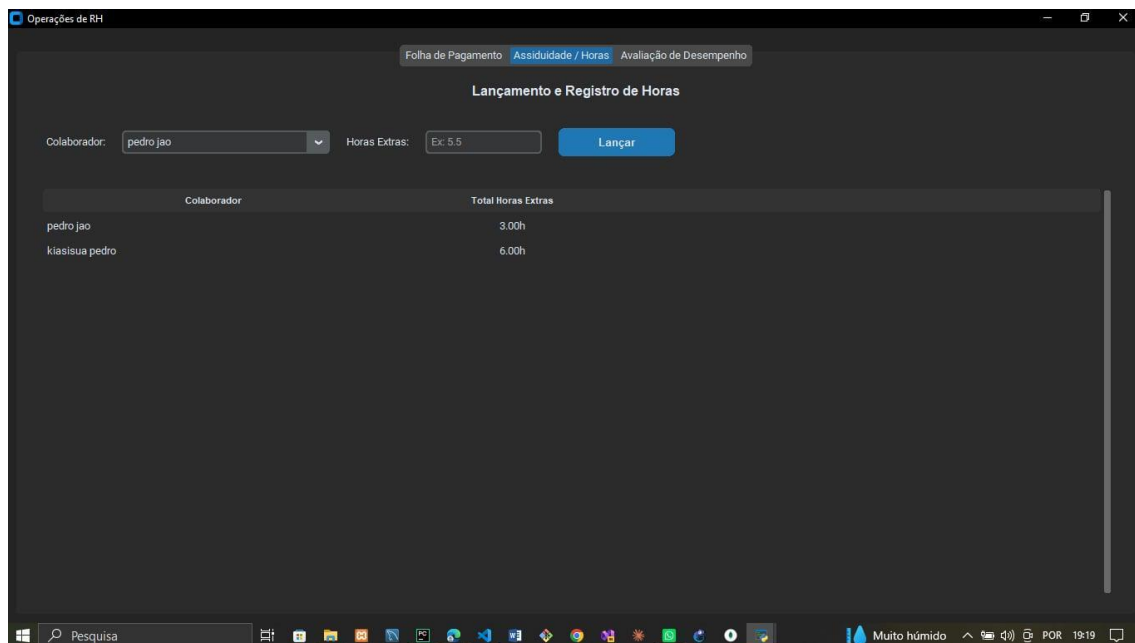


Figura 5. TELA DE LANÇAMENTOS DE REGISTOS

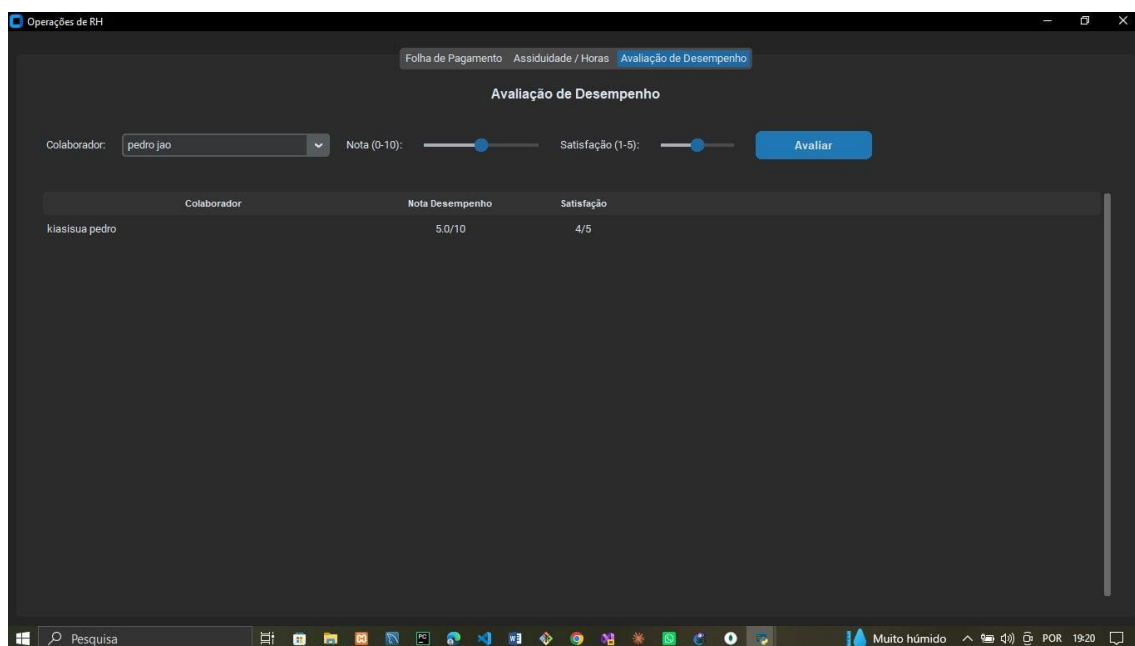


Figura 6. TELA DE AVALIAÇÃO E DESEMPENHO